University of Calgary
CPSC 433 – Artificial Intelligence

# Course Timetabling via Three Search Paradigms

OR-Tree Feasibility, Set-Based Improvement, and AND-Tree Optimization

Formal Search Models and Processes – Proposal


Member 1: Syed Essam Uddin Khawaja 30242943

Member 2: [NAME / UCID]

Member 3: [NAME / UCID]

Member 4: [NAME / UCID]

Member 5: [NAME / UCID]


Date: [SUBMISSION DATE]

# Contents

**1 Introduction** **3**

1.1 Scheduling problem and motivation . . . 3

1.2 Models proposed . . . 3

**2 Formal Scheduling Problem** **3**

2.1 Components, slots, and domains . . . 3

2.2 Hard constraints . . . 3

2.3 Soft constraints and objective . . . 4

**3 Overall Search Architecture** **4**

3.1 How the models relate . . . 4

**4 Model I: OR-Tree Feasibility Search** **4**

4.1 Search model: $A_{\vee} = (S_{\vee}, T_{\vee})$ . . . 4

4.2 Search process: $P_{\vee} = (A_{\vee}, Env_{\vee}, K_{\vee})$ . . . 5

4.3 Search instance: $Inst_{\vee} = (q_0, G_{\vee})$ . . . 5

4.4 Algorithm 1: OR-tree feasibility . . . 6

4.5 Worked OR-tree derivation . . . 6

**5 Model II: Set-Based Local Optimization** **6**

5.1 Search model: $A_{set} = (S_{set}, T_{set})$ . . . 6

5.2 Search process: $P_{set} = (A_{set}, Env_{set}, K_{set})$ . . . 7

5.3 Search instance: $Inst_{set} = (q_0, G_{set})$ . . . 7

5.4 Algorithm 2: Set-based hill climbing . . . 8

5.5 Worked set-based derivation . . . 8

**6 Model III: AND-Tree Optimization** **8**

6.1 Motivation and difference from OR-tree . . . 8

6.2 Search model: $A_{\wedge} = (S_{\wedge}, T_{\wedge})$ . . . 8

6.3 Search process: $P_{\wedge} = (A_{\wedge}, Env_{\wedge}, K_{\wedge})$ . . . 9

6.4 Search instance: $Inst_{\wedge} = (q_0, G_{\wedge})$ . . . 10

6.5 Algorithm 3: AND-tree branch-and-bound . . . 10

6.6 Worked AND-tree derivation . . . 10

**7 End-to-End Integration and Comparison** **11**

7.1 Algorithm 4: Optional combined pipeline . . . . . 11
7.2 Comparison of the three paradigms . . . . . 11
7.3 Unified worked example . . . . . 11

**8 Discussion and Limitations** **11**

**9 Conclusion** **12**

**A Complete Constraint Formulas** **12**

**B Additional Derivations and Search Trees** **12**

**Abstract**

[**TO FILL:** Write 150–200 words: problem, all three paradigms, different state representations, search controls, and the benefit/limitation of each. Write this last.]

# 1 Introduction

## 1.1 Scheduling problem and motivation

[**TO FILL:** Describe the University of Calgary weekly lecture/tutorial scheduling problem. Distinguish hard constraints from soft penalties.]

## 1.2 Models proposed

We consider three search paradigms:

- **OR-tree**: find one feasible timetable.
- **Set-based**: improve a feasible timetable through local changes.
- **AND-tree**: systematically explore scheduling decisions to seek a globally optimal timetable, using branch-and-bound.

[**TO FILL:** Explain whether the three models will be proposed as independent alternatives, a sequential hybrid, or both. State clearly what the instructor would select to implement.]

# 2 Formal Scheduling Problem

## 2.1 Components, slots, and domains

Let $L$ be the set of lectures, $T$ the set of tutorials/labs, and $X = L \cup T$ the full set of schedulable components. Let $S$ be the weekly slot set, and $D(x) \subseteq S$ the allowed slots for component $x \in X$. A complete timetable is a function

$$a : X \to S, \qquad a(x) \in D(x) \text{ for every } x \in X. \tag{1}$$

A *partial assignment* $a$ is defined only on $\mathrm{dom}(a) \subseteq X$.

[**TO FILL:** Specify exact MWF, TR, MW, and Friday slot patterns; distinguish lecture and tutorial domains. Explain abstract weekly slots and real-time overlap.]

## 2.2 Hard constraints

Let $C_1, \ldots, C_h$ be the required hard constraints. Define

$$\boxed{\mathrm{Const}(a) = \bigwedge_{i=1}^{h} C_i(a)}. \tag{2}$$

For partial assignments, Consistent($a$) means none of the hard constraints that can already be evaluated is violated.

[**TO FILL:** Formalize all required constraints: lecture/tutorial capacities, active-learning limits, lecture–tutorial compatibility, not-compatible pairs, preassignments, unwanted slots, LEC 9 evening, 5XX non-overlap, TR 11:00–12:30 restriction, and conditional CPSC 851/913 bookings. State how shared tutorials are interpreted.]

### 2.3 Soft constraints and objective

Let the five penalty terms represent minimum lecture occupancy, minimum tutorial occupancy, preferences, requested pairings, and same-course lecture sections:

$$\boxed{\begin{aligned}\text{Eval}(a) = E_{L\,\min}(a) + E_{T\,\min}(a) + E_{\text{pref}}(a) \\ + E_{\text{pair}}(a) + E_{\text{section}}(a).\end{aligned}} \tag{3}$$

The optimization problem is

$$\boxed{\min_{a} \text{Eval}(a) \quad \text{subject to} \;\; \text{Const}(a) = \mathsf{true}.} \tag{4}$$

[**TO FILL:** Give all five penalty formulas with their input parameters. Note that penalties are nonnegative.]

## 3 Overall Search Architecture

### 3.1 How the models relate

Our optional combined pipeline is

$$\boxed{\begin{array}{c}\text{OR-tree} \longrightarrow a_{\text{feasible}} \\ \downarrow \\ \text{Set-based improvement} \longrightarrow a_{\text{best}} \\ \downarrow \\ \text{AND-tree branch-and-bound, initialized with } a_{\text{best}}.\end{array}} \tag{5}$$

The AND-tree explores the *original scheduling choices*, not merely the local neighbors visited by set-based search. It can establish optimality only if its search completes with sound pruning.

[**TO FILL:** Discuss the option to run OR-tree + Set-based without exact AND-tree optimization, or OR-tree + AND-tree directly. Distinguish the three paradigms in their own right.]

## 4 Model I: OR-Tree Feasibility Search

### 4.1 Search model: $A_\vee = (S_\vee, T_\vee)$

**Problem descriptions.**

$$\boxed{\mathit{Prob}_\vee = \{(a, U) : a \text{ is partial},\; U = X \setminus \text{dom}(a)\}.} \tag{6}$$

Here $a$ is the current partial timetable, and $U$ contains unscheduled components. A node has the lecture's form

$$((a, U), sol), \qquad sol \in \{\mathsf{yes}, ?, \mathsf{no}\}. \tag{7}$$

**Alternatives.** Select a component $x \in U$ and define

$$D_a(x) = \{s \in D(x) : \text{Consistent}(a \cup \{x \mapsto s\})\}. \tag{8}$$

For every $s_i \in D_a(x)$, let

$$pr_i = (a \cup \{x \mapsto s_i\}, U \setminus \{x\}). \tag{9}$$

Then the application-specific alternative relation is

$$\boxed{Altern(pr, pr_1, \ldots, pr_r), \qquad r = |D_a(x)|.} \tag{10}$$

**States and transitions.** Following Lecture 11,

$$S_\vee \subseteq Otree, \qquad T_\vee = \{(q, q') : Erw_\vee(q, q') \text{ or } Erw^*_\vee(q, q')\}. \tag{11}$$

The extension relation marks a solved leaf yes, marks an unsolvable leaf no, or expands a leaf using *Altern*. Extensions can act on leaves inside a larger tree.

[**TO FILL:** Write the four formal $Erw_\vee$ cases from Lecture 11, using the definitions above.]

### 4.2 Search process: $P_\vee = (A_\vee, Env_\vee, K_\vee)$

[**TO FILL:** Define $Env_\vee$, leaf selection, and slot ordering. Proposed $K_\vee$: depth-first unfinished-leaf selection and MRV component selection. No forward checking is necessary for correctness.]

The MRV component selection is

$$\boxed{x^* = \arg\min_{x \in U} |D_a(x)|.} \tag{12}$$

### 4.3 Search instance: $Inst_\vee = (q_0, G_\vee)$

Start with

$$a_0 = \varnothing, \quad U_0 = X, \quad q_0 = ((a_0, U_0), ?). \tag{13}$$

A leaf is solved if $U = \varnothing$ and Const$(a)$ is true. The search halts when a solved branch exists or all alternatives have failed; these are distinct outcomes.

[**TO FILL:** Define $G_\vee$ recursively in terms of the tree and distinguish success from exhaustive failure.]

### 4.4 Algorithm 1: OR-tree feasibility

**Algorithm 1** OR-Tree Feasibility Search (operational outline)

**Require:** Components $X$, domains $D$, partial checker Consistent
**Ensure:** Complete valid timetable, or FAILURE

Initialize tree $q \leftarrow ((\varnothing, X), ?)$
**while** an unresolved leaf exists **do**
  $v \leftarrow$ deepest unresolved leaf of $q$
  $(a, U) \leftarrow$ problem description at $v$
  **if** $U = \varnothing$ **then**
    **if** Const$(a)$ **then**
      Mark $v$ as yes; **return** $a$
    **else**
      Mark $v$ as no;
      **continue**
    **end if**
  **end if**
  $x \leftarrow \arg\min_{y \in U} |D_a(y)|$ ▷ MRV
  $C \leftarrow D_a(x)$
  **if** $C = \varnothing$ **then**
    Mark $v$ as no
  **else**
    **for all** $s \in C$ in deterministic slot order **do**
      Add child $((a \cup \{x \mapsto s\}, U \setminus \{x\}), ?)$ to $v$
    **end for**
  **end if**
**end while**
**return** FAILURE

### 4.5 Worked OR-tree derivation

[**TO FILL:** Starting with $a_0 = \varnothing$ and $U_0 = X$, show every intermediate $(a_i, U_i)$; draw an OR-tree with at least one failing alternative and one solved leaf. Explain why the search stops on its first successful path.]

## 5 Model II: Set-Based Local Optimization

### 5.1 Search model: $A_{set} = (S_{set}, T_{set})$

**Facts.**

$$\boxed{F = \{\text{Assigned}(x, s) : x \in X,\ s \in D(x)\}.} \tag{14}$$

The fact Assigned$(x, s)$ means that component $x$ occupies weekly slot $s$.

**States.**

$$\boxed{S_{set} = \{q \subseteq F : q \text{ encodes one complete valid timetable}\} \subseteq 2^F.} \tag{15}$$

For any $q \in S_{set}$, let $a_q$ be the complete timetable that its facts represent.

**Extensions.**

$$\begin{aligned} &\text{Move: } \{\text{Assigned}(x, s_1)\} \longrightarrow \{\text{Assigned}(x, s_2)\}; \\ &\text{Swap: } \{\text{Assigned}(x, s_1), \text{Assigned}(y, s_2)\} \\ &\qquad\qquad \longrightarrow \{\text{Assigned}(x, s_2), \text{Assigned}(y, s_1)\}. \end{aligned} \tag{16}$$

Here $x, y$ are distinct components and $s_1, s_2$ are their current slots. The rules are applicable only if domains, preassignments, and all hard constraints are respected in the resulting timetable.

$$\boxed{q' = (q \setminus A) \cup B, \qquad A \to B \in EXT.} \tag{17}$$

$$T_{set} = \{(q, q') : A \to B \in EXT,\ A \subseteq q,\ q' = (q \setminus A) \cup B,\ q' \in S_{set}\}. \tag{18}$$

## 5.2 Search process: $P_{set} = (A_{set}, Env_{set}, K_{set})$

The proposed greedy weight function is

$$\boxed{f_{weight}(A, B, e) = \text{Eval}(a_{(q \setminus A) \cup B}),} \tag{19}$$

where the environment $e$ provides the current timetable $q$ and other problem information. The function $f_{select}$ breaks ties between equally weighted extensions.

[**TO FILL:** Define $Env_{set}$, $f_{select}$, the strict-improvement condition, and the neighborhood $N(q)$. Explain the possibility of getting stuck at a local optimum.]

## 5.3 Search instance: $Inst_{set} = (q_0, G_{set})$

The OR-tree produces $a_{\text{feasible}}$. The set-based starting state is

$$\boxed{q_0 = \{\text{Assigned}(x, a_{\text{feasible}}(x)) : x \in X\}.} \tag{20}$$

[**TO FILL:** Define a practical goal/stopping condition: score zero, no improving neighbor, or a specified search budget.]

### 5.4 Algorithm 2: Set-based hill climbing

**Algorithm 2** Set-Based Optimization

**Require:** Feasible timetable $a_{\text{feasible}}$, $EXT$, Eval
**Ensure:** Improved valid timetable

$q \leftarrow \{\text{Assigned}(x, a_{\text{feasible}}(x)) : x \in X\}$
**while** $\text{Eval}(a_q) > 0$ **do**
  $N \leftarrow \{q' : (q, q') \in T_{set},\ q' \neq q\}$
  **if** $N = \varnothing$ **then**
    **return** $a_q$
  **end if**
  $m \leftarrow \min_{q' \in N} \text{Eval}(a_{q'})$
  **if** $m \geq \text{Eval}(a_q)$ **then**
    **return** $a_q$ ▷ Local optimum
  **end if**
  $q \leftarrow f_{select}(\{q' \in N : \text{Eval}(a_{q'}) = m\})$
**end while**
**return** $a_q$

### 5.5 Worked set-based derivation

[**TO FILL:** Represent the OR-tree solution as facts. Show Move and optionally Swap using $q' = (q \setminus A) \cup B$. Give before/after evaluation values and verify $\text{Const}(a_{q'})$.]

## 6 Model III: AND-Tree Optimization

### 6.1 Motivation and difference from OR-tree

An OR-tree requires *one* successful alternative. An AND-tree requires the chosen division's subproblems to *all* be solved and mutually compatible. For optimization, different divisions must also be compared to find the best result. The AND-tree may backtrack to another division if the current one fails or cannot beat an existing solution.

[**TO FILL:** Explain why AND-tree plus branch-and-bound is a reasonable alternative to local set-based optimization. Cite Lecture 09.]

### 6.2 Search model: $A_\wedge = (S_\wedge, T_\wedge)$

**Problem descriptions.** We propose two kinds of problem description:

$$\boxed{Prob_\wedge = \{\mathsf{Schedule}(a, U)\} \ \cup \ \{\mathsf{Verify}(a, x, s)\}.} \tag{21}$$

Here $\mathsf{Schedule}(a, U)$ asks for a completion of partial assignment $a$ on the remaining components $U$. The problem $\mathsf{Verify}(a, x, s)$ asks whether putting component $x$ in slot $s$ is consistent with the current timetable $a$.

**Division relation.** Select $x \in U$ and a candidate $s \in D(x)$. One possible division is

$$\boxed{\begin{array}{l} \mathit{Div}(\mathsf{Schedule}(a, U), \\ \quad \mathsf{Verify}(a, x, s), \\ \quad \mathsf{Schedule}(a \cup \{x \mapsto s\}, U \setminus \{x\})). \end{array}} \tag{22}$$

The division has *two necessary subproblems*: check the new assignment, and schedule everything remaining with the new assignment fixed. A different candidate $s$ produces a different possible *Div* choice. Thus this is an AND division, not an OR-tree *Altern* list.

**Solved / failed subproblems.**

$$\mathsf{Verify}(a, x, s) \text{ is solved} \iff \mathrm{Consistent}(a \cup \{x \mapsto s\}), \tag{23}$$

$$\mathsf{Schedule}(a, \varnothing) \text{ is solved} \iff \mathrm{Const}(a) = \mathsf{true}. \tag{24}$$

If verification fails, that division cannot succeed. The controller must try a different division by the lecture's AND-tree backtracking mechanism.

**States and transitions.** Lecture 09 defines

$$\boxed{S_\wedge \subseteq \mathit{Atree}, \quad \mathit{sol} \in \{\mathsf{yes}, ?\}.} \tag{25}$$

An AND-tree state is an entire tree of subproblems, not a single partial assignment. Its transitions follow the lecture's expansion and contraction relations:

$$\boxed{T_\wedge = \{(q, q') : \mathit{Erw}_\wedge(q, q') \text{ or } \mathit{Erw}^*_\wedge(q, q')\}.} \tag{26}$$

[**TO FILL:** State the formal AND-tree $\mathit{Erw}_\wedge$ and $\mathit{Erw}^*_\wedge$ rules from Lecture 09. Explain why AND-trees use yes and ? rather than an explicit no node status, and how contraction/backtracking handles failed divisions.]

## 6.3 Search process: $P_\wedge = (A_\wedge, \mathit{Env}_\wedge, K_\wedge)$

We propose MRV for selecting the next component and branch-and-bound for choosing/pruning divisions. The environment records the best complete timetable found so far, $a_{\mathsf{Best}}$, and its score

$$\boxed{B = \mathrm{Eval}(a_{\mathsf{Best}}).} \tag{27}$$

The valid timetable returned by OR-tree (or improved by set-based search) provides the initial incumbent $a_{\mathsf{Best}}$. For a partial timetable $a$, let $\mathsf{LB}(a, U)$ be a sound lower bound on the penalty of *every complete valid extension* of $a$. Then

$$\boxed{\mathsf{LB}(a, U) \geq B \quad \Longrightarrow \quad \text{prune this division.}} \tag{28}$$

The lower bound must satisfy

$$\boxed{\mathsf{LB}(a, U) \leq \mathrm{Eval}(\widehat{a})} \tag{29}$$

for every feasible completion $\widehat{a}$ of $a$.

[**TO FILL:** Define a concrete admissible lower bound. A safe initial bound is the sum of nonnegative preference and pair/section penalties already made unavoidable by assigned components; omit uncertain minimum-occupancy penalties. Explain how $f_{leaf}$ and $f_{trans}$ select a leaf and a division.]

### 6.4 Search instance: $Inst_{\wedge} = (q_0, G_{\wedge})$

Begin the AND-tree optimization from the original problem, but pass the best known complete timetable as a bound:

$$\boxed{q_0 = (\mathsf{Schedule}(\varnothing, X), ?).} \tag{30}$$

[**TO FILL:** Define $G_{\wedge}$ using Lecture 09: root solved, all subproblems solved compatibly, or every applicable division/backtracking option exhausted. Specify separately what certifies optimality: all better candidates have been examined or soundly pruned.]

### 6.5 Algorithm 3: AND-tree branch-and-bound

**Algorithm 3** AND-Tree Optimization (control implementing *Div*)

**Require:** Original CSP, a complete valid incumbent $a_{\mathsf{Best}}$
**Ensure:** Best valid schedule found; optimal if exhaustive

$B \leftarrow \mathrm{Eval}(a_{\mathsf{Best}})$
**procedure** OPTIMIZE($a, U$)
  **if** $\mathsf{LB}(a, U) \geq B$ **then**
    **return** ▷ Soundly pruned division
  **end if**
  **if** $U = \varnothing$ **then**
    **if** $\mathrm{Const}(a)$ **and** $\mathrm{Eval}(a) < B$ **then**
      $a_{\mathsf{Best}} \leftarrow a$; $B \leftarrow \mathrm{Eval}(a)$
    **end if**
    **return**
  **end if**
  $x \leftarrow \arg\min_{y \in U} |D_a(y)|$ ▷ MRV
  **for all** $s \in D(x)$ **do**
    Consider division $Div(\mathsf{Schedule}(a, U),$
      $\mathsf{Verify}(a, x, s), \mathsf{Schedule}(a \cup \{x \mapsto s\}, U \setminus \{x\}))$
    **if** $\mathrm{Consistent}(a \cup \{x \mapsto s\})$ **then** ▷ Verify child
      OPTIMIZE($a \cup \{x \mapsto s\}, U \setminus \{x\}$) ▷ Remaining child
    **end if**
    ▷ If needed, try alternate *Div* (AND-tree backtracking)
  **end for**
**end procedure**
OPTIMIZE($\varnothing, X$)
**return** $a_{\mathsf{Best}}$ and completion/optimality status

This is operational pseudocode for the branch-and-bound control. The formal model remains an AND-tree with *Div* and $Erw^*_{\wedge}$; the recursive procedure implements selecting divisions and exploring them. It is not an OR-tree *Altern* relation.

### 6.6 Worked AND-tree derivation

[**TO FILL:** Using the same small timetable instance, show at least two possible *Div* choices for LEC 02: MWF 9–10 and MWF 10–11. Expand one AND division into its Verify and Schedule children.

Show how the incumbent penalty is updated or how a division is pruned. If the starting incumbent has score zero and penalties are nonnegative, explain why that already certifies global optimality.]

# 7 End-to-End Integration and Comparison

## 7.1 Algorithm 4: Optional combined pipeline

**Algorithm 4** Combined Three-Paradigm Pipeline

**Require:** Scheduling instance $I$
**Ensure:** Best valid timetable found, or FAILURE
1: $a_0 \leftarrow \text{ORFeasibility}(I)$
2: **if** $a_0 = \text{FAILURE}$ **then**
3: **return** FAILURE
4: **end if**
5: $a_1 \leftarrow \text{SetBasedImprove}(I, a_0)$
6: $a_2 \leftarrow \text{ANDOptimize}(I, a_1)$
7: **return** $a_2$ ▷ Optimal only if AND stage completed

## 7.2 Comparison of the three paradigms

Table 1: Summary of proposed models

| | OR-tree | Set-based | AND-tree |
|---|---|---|---|
| Purpose | Feasibility | Local improvement | Exact optimization |
| State | OR-tree | Assignment fact set | AND-tree |
| Primitive | *Altern* | *EXT* | *Div* |
| Control | DFS + MRV | Hill climbing | Branch-and-bound |
| Optimal? | Not targeted | Not guaranteed | If complete |

[**TO FILL:** Explain each model's pros and cons: search memory, runtime, completeness, and solution quality. Avoid claiming an AND-tree guarantees optimality if the search budget expires.]

## 7.3 Unified worked example

[**TO FILL:** Use one small course-scheduling instance across all three models. Show the OR-tree's first feasible score, set-based's improved score, and AND-tree's exploration/proof of optimality. The same input and penalty parameters must be used in all three worked examples.]

# 8 Discussion and Limitations

[**TO FILL:** Explain why three models may be informative but more complex than required. Discuss infeasible inputs, local minima, sound bounding, valid outputs on timeout, and algorithm selection for large instances.]

## 9 Conclusion

[**TO FILL:** Summarize the three models, their distinct state data structures, and why the proposed architecture is appropriate for the assignment. Do not claim experimentally verified performance before testing.]

## References

[1] CPSC 433. *Project Description*, Version 1, University of Calgary, Fall 2025.

[2] CPSC 433. *Project Problem Description*, Version 1, University of Calgary, Fall 2025.

[3] CPSC 433. *Lecture 05: Search*, University of Calgary.

[4] CPSC 433. *Lecture 06: Set-Based Search*, University of Calgary.

[5] CPSC 433. *Lecture 09: And-Tree-Based Search*, University of Calgary.

[6] CPSC 433. *Lecture 11: Or-Tree-Based Search*, University of Calgary.

[7] CPSC 433. *Lecture 14: Search Controls*, University of Calgary.

## A Complete Constraint Formulas

[**TO FILL:** Optional: extended detailed formalization of all hard and soft constraints.]

## B Additional Derivations and Search Trees

[**TO FILL:** Optional: extra OR-tree, AND-tree, and set-based state traces.]